

Refactoring to make code testable

Before commencing testing on our code, we realized that our code wasn't very testable. By that, we mean that most methods were private, and we had a very big 'God' class that contained a lot of methods that weren't necessarily related to that class. We decided that the first step of this phase should be refactoring the Board class to make testing more straightforward.

We removed a couple of unused fields, such as BOARD, which was a 2d array we never used, and NUM_COINS, which was, again, never used and not needed because the number of coins is constant. We also removed unused methods: "tick" method that was in Entity.java, because this was being handled by a tick method in MainCharacter.java (restricts where the main character can move); get_coordinates method in MainCharacter.java because the entity superclass already has a method implemented to return entity coordinates. We also renamed a couple of methods to ensure clarity.

We then moved around methods that should be in their own respective classes instead of Board.java. We moved the concord timer that sets the speed of the concord officer to the ConcordOfficer.java class; we created a new class for gametimer and moved its respective methods into that class; we put a draw method in score instead of calling a drawScore method in Board.java, so that score draws itself; we created a getImage function in Entity.java that takes the file name as a parameter to avoid rewriting try and catch statements all the time; we moved LostNote appearance logic into LostNote.java from Board.java; we made board a field of Entity so that we didn't have to pass board as a parameter for every function, but just the constructors; we moved collectCoins and collectLostNote to MainCharacter.java from Board.java because the main character is the entity that does the collecting of rewards; we moved isCollidingParkedCar and isCollidingCone to MainCharacter.java from board because that information stops the main character from moving.

We then separated larger methods into smaller methods to increase code testability. We created initBoard and populateBoard methods in Board.java that are called in the constructor instead of having unreadable code in the constructor. We also created some getter functions for better encapsulation, such as updateCoinsCollected in Board.java.

Unit Tests

The first tests we wrote were constructor tests for all classes. We started with screen tests. Writing tests for just the construction of these objects for the most part resulted in 100% line coverage of the respective screen class, except for the Screen and Instructions test classes. The exceptions to this were event handlers which Dr. Alimadadi told us not to test, and catch statements for errors in loading certain

images. Screen, EndScreen, and Lose Screen didn't receive constructor tests because they're abstract classes and, therefore tested through their subclasses.

We then created constructor tests for all other .java files. This resulted in a corresponding test file for every src file. We were unable to write constructor tests for abstract fields. After writing all the constructor tests, our code coverage was relatively high (around 80%). We then moved on to integration tests by observing which lines were missed in certain files and created tests considering the functionality of the program.

Integration Tests

We started integration tests by thinking about interactions that are important to the functionality of our game. We wrote tests for verifying main character movement on the board - collision with concord officer; collision with cones; collision with parked cars; collision with borders; collision with coins; collision with lost notes; and verifying game ends when certain tiles are reached with the needed coins collected.

We also wrote tests for the delay of the concord officer and verifying the timer was updating correctly. Regarding structural tests, we developed test cases that covered as much implemented behaviour as possible. For instance, we created tests that test the Concord officer movement with the cones and parked car that will revert its position if it does. This test ensures that when this happens it does not create any errors within the code. Another test that tests the behaviour of our implementation of code is a test of the concord officer hitting the main character. This test checks the branch coverage of the interaction between the enemy and the player. We also tested the case where it does not collide with the player. After integrating many structural tests we saw that our branch coverage resulted in an increase to 91%. This significantly increases our reliability of the game. It ensures more decision points in the code are tested and reduces the risk of undetected logic errors. It also strengthens our code quality by covering different execution paths. Which ultimately makes our code maintainable for future changes.

In the MainCharacter class, methods like tick, keyReleased, and delayedMove manage the player movement, game state updates, and event handling. Test cases that test the movement of the main character ensure player movement updates correctly based on the input. Collision detection methods ensure that the player cannot move through the cones and parked cars. Test cases like testCollision with cones and parked cars reinforce these behaviours. Concord officer movement towards the main character is controlled through findPathDijkstra method. Tests like testOfficerCollisionWithMainCharacter ensure that the concord officer follows their path and correctly chases the player.

Other integration tests, such as testCollectCoins and testCollectLostNote confirmed that when the main character interacted with collectible items, the relevant ArrayLists

in the Board were updated and the score adjusted accordingly. These tests required proper coordination between the main character, entity ArrayLists, and UI update mechanisms. Additionally, we developed tests to cover game constraints, such as board boundaries and movement restrictions within the entrance and exit cells. These tests ensured that the main character would stay within the proper bounds.

Findings

Originally, there was a test class for each class in our source/production code. Some of the screen classes were not tested because they are abstract classes. Thus, we removed the following test classes/files: EndScreenTest, LoseScreenTest, ScreenTest, and StaticEntityTest. Some of the test classes tested a certain class multiple times. For example, there were tests for both the Entity class and the Main Character class. The EntityTest class focused on using the constructors of the different types of Entities. The individual test classes for each of the specific Entities were created to test the methods that were more specific to that subclass of Entity. For example, the move method for the MainCharacter object was tested a lot, and these tests were added into the MainCharacterTest class because these tests were specific to the MainCharacter object.

We fixed a bug where if the user holds down one of the valid movement keys (W,A,S,D), the main character would pause for a split second, and then would proceed to move rapidly. We fixed this issue by using the keyReleased function and creating a boolean called key_pressed. We also fixed a java.util.ConcurrentModificationException, which was occurring when the static entities such as coins and lostnotes, were being collected from the Board by the main character. This occurred because we were originally iterating through an arraylist at the same time as removing an element from the arraylist. We fixed this bug by using an Iterator which has a remove method to safely remove an element while iterating through that arraylist.

Test Quality and Coverage

To ensure the quality of our test cases, we made sure that each test had 1 clear objective and we named our tests in a way to clearly convey what the test case was testing. We made several different test classes to make sure there was a separation of concern with respect to the test cases.

We used Jacoco for code coverage during Phase 3. Our overall branch coverage is 25% for the screen tests in our screen package, and 94% for our tests in our game package. In the game package, there were some classes that had “n/a” under the “Missed Branches” section. This is because these classes did not have any statements that would cause several possible branches to occur. For example, classes that do not have “if” nor “if-else” statements would show that there are no branches that can be “missed”. We also tested multiple edge cases, even if those

cases were not possible given the game logic. This was done to ensure maximum branch coverage.

Most of the classes in the game package of our program had 100% instruction coverage or were close to 100%; The Entity class had 88% instruction coverage, and Concord Officer had 99% instruction coverage. The Board class has a low instruction coverage, which is 64% because several “drawing” methods were not tested. The draw methods are considered UI tests, and this was not covered (also, Dr. Alimadadi mentioned that the only UI tests to be included for Phase 3 would be testing the creation of screens). For UI tests, we did test the creation of screens by using the constructors. Several of the draw methods for drawing entities onto the Board object were private classes so they were not tested as it is common to not test private methods. We did not want to make these private classes protected (changing the access modifier from “private” to “protected”) just to allow us to test them. We know that since the Board was populated correctly, these private “draw” methods work.

In the screen package, all of the classes, except for the Screen and Instructions classes, have 100% instruction coverage. The Screen class had 77% in instruction coverage and the Instructions class had 87% in instruction coverage. The Screen, MainMenu, and Instructions classes are the only classes that have branch coverage to be reported (all other classes in the screen package have “n/a” for the number of branches missed. Since most of the methods within the Screen, Instructions, and MainMenu classes are UI methods, the branch coverage for these classes will be low. The paintComponent method in the MainMenu class had 50% branch coverage because the method does not do anything if the backgroundImage is null, but this method was intended to be this way. Therefore, the overall branch coverage of the entire screen package is 25% as this screen package contained many UI-focused methods.

For the overall line coverage of the screen package, there were 24 missed lines out of a total of 155 lines. Thus, the line coverage of the screen package is approximately 85%. Only the Screen and Instructions classes had lines missed; The Screen class had 20 lines missed out of 59 lines, and the Instructions class had 4 missed lines out of 34 total lines. Since screen had more lines, and is the abstract class for all screen types, it was harder to actually test the lines of source code.

The overall branch coverage of the game package is 94%. The lowest branch coverage was from the Board class since many of its private draw (UI) methods were not tested. The StaticEntity class and its subclasses did not have any branches to be missed and, therefore, had “n/a” missed branches. The ConcordOfficer and the MainCharacter were more complex classes in terms of their implementations and they had many branches within their implementation, so we tried to get as many branches covered during testing. The game package also had 90% in instruction

coverage, where the Board class had the most instructions, but also had the lowest instruction coverage of 64% (again, due to the many private drawing/UI methods).

For the game package, the overall line coverage was about 87% since there were 60 lines missed out of 478 lines total. The StaticEntity class and its subclasses all had 100% line coverage. There are only 4 classes that do not have 100% line coverage, which includes the following classes (and their respective approximate line coverage in brackets): Board (62%), Main (0%), Entity (86%), ConcordOfficer (98%). It was easier to test non-static methods (ex. The main method was difficult to test since it is static). The results showed that it was easier to test classes that are not abstract and not static as they tended to have more line coverage (for example Concord Officer class had a higher line coverage than the Entity class, and the Main class had 0% line coverage because it contains a static main method). Also, classes that had fewer UI methods made it easier to test and had higher line coverage.

We tested each case where the Main Character object interacted with all the other Entities (ex. Collisions with the concord officer, cones, parked cars, borders, coins etc.). This was especially important because the requirements of the game specified exactly how the Main Character should interact when it collides with other Entities on the Board. This is also why the MainCharacterTest class has the most test cases compared to all the other test classes. Most of the methods that were not tested were methods that drew Entities and images onto the Board. UI testing was not covered in class, however, functional and structural tests were taught and used in Phase 3.

We also ran into a problem where the true && false branch of the checkGameEnd() method in the Board class was impossible to cover because tick() was being called before checkGameEnd() in actionPerformed(). This caused an issue where, if the main character was manually positioned at (14, Y) in a test, the call to tick() would immediately clamp the coordinates back to (13, Y), since (14, Y) is out of bounds (except for (14, 13) which is the exit cell). We fixed this by reordering the Tick() and checkGameEnd() method calls within actionPerformed(), which ensures that the position is evaluated before any coordinate correction happens. As a result, we were able to increase the branch coverage of the Board from 79% to 81%.